DESIGN OF ENTITY RELATIONSHIP (ER) DIAGRAM

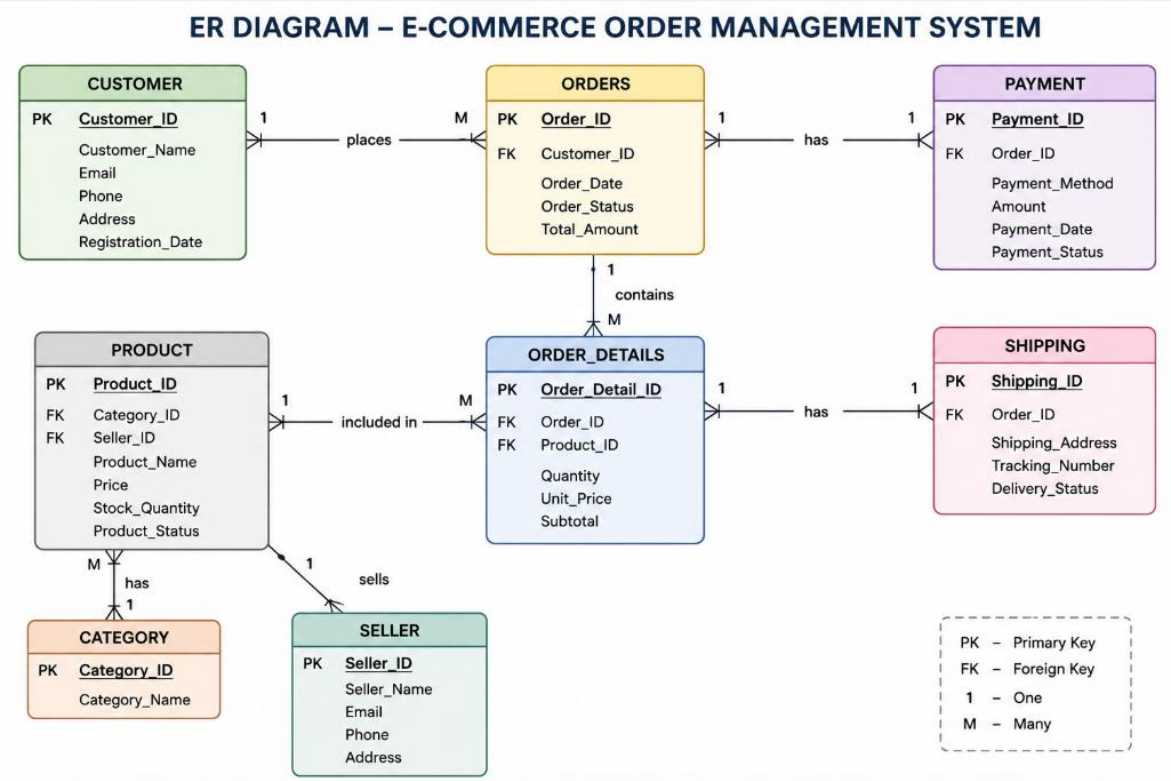
The ER Diagram represents the major entities of the E-Commerce Order Management System and the relationships between them. Each entity has a **Primary Key (PK)** for unique identification, while **Foreign Keys (FK)** are used to establish relationships between tables.

Entities and Keys

Entity	Primary Key	Important Foreign Keys
Customer	Customer_ID	—
Seller	Seller_ID	—
Category	Category_ID	—
Product	Product_ID	Category_ID, Seller_ID
Orders	Order_ID	Customer_ID
Order_Details	Order_Detail_ID	Order_ID, Product_ID
Payment	Payment_ID	Order_ID
Shipping	Shipping_ID	Order_ID

The Product entity connects with Category and Seller, while Order_Details acts as an associative entity between Orders and Products.

ER Diagram



The main relationships are **Customer → Orders (1:M)**, **Seller → Product (1:M)**, **Category → Product (1:M)**, **Orders → Order_Details (1:M)**, **Product → Order_Details (1:M)**, and Orders associated with Payment and Shipping.

Conversion of ER Diagram into Relational Schema

The ER Diagram is converted into relational tables. Each entity becomes a relation, its attributes become columns, and relationships are represented using foreign keys.

CUSTOMER(

Customer_ID PK,

Customer_Name,

Email,

Phone,

Address,

Registration_Date

)

SELLER(

Seller_ID PK,

Seller_Name,

Email,

Phone,

Address

)

CATEGORY(

Category_ID PK,

Category_Name

)

PRODUCT(

Product_ID PK,

Product_Name,

Category_ID FK,

Seller_ID FK,

Price,

Stock_Quantity,

Product_Status

)

ORDERS(

Order_ID PK,

Customer_ID FK,

Order_Date,

Order_Status,

Total_Amount

)

ORDER_DETAILS(

Order_Detail_ID PK,

Order_ID FK,

Product_ID FK,

Quantity,

Unit_Price,

Subtotal

)

PAYMENT(

Payment_ID PK,

Order_ID FK,

Payment_Method,

Amount,

Payment_Date,

Payment_Status

)

SHIPPING(

Shipping_ID PK,

Order_ID FK,

Shipping_Address,

Tracking_Number,

Delivery_Status

)

Relationship Representation

CUSTOMER (1) ———< places >——— (M) ORDERS

SELLER (1) ————< sells >———— (M) PRODUCT

CATEGORY (1) ———< contains >——— (M) PRODUCT

ORDERS (1) ————< contains >——— (M) ORDER_DETAILS

PRODUCT (1) ————< appears in >—— (M) ORDER_DETAILS

ORDERS (1) ———— has ———————— (1) PAYMENT

ORDERS (1) ———— has ———————— (1) SHIPPING

SQL query to create all tables

USE Ecommerce_DB;

1. Customer Table

```
CREATE TABLE Customer (
   Customer_ID INT PRIMARY KEY,
   Customer_Name VARCHAR(100) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   Phone VARCHAR(15),
   Address VARCHAR(255),
   Registration_Date DATE
);
```

2. Seller Table

```
CREATE TABLE Seller (
  Seller_ID INT PRIMARY KEY,
  Seller_Name VARCHAR(100) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Phone VARCHAR(15),
  Address VARCHAR(255)
);
```

3. Category Table

```
CREATE TABLE Category (
  Category_ID INT PRIMARY KEY,
  Category_Name VARCHAR(100) NOT NULL
);
```

4. Product Table

```
CREATE TABLE Product (
  Product_ID INT PRIMARY KEY,
  Product_Name VARCHAR(100) NOT NULL,
  Category_ID INT,
  Seller_ID INT,
  Price DECIMAL(10,2),
  Stock_Quantity INT,
  Product_Status VARCHAR(50),

  FOREIGN KEY (Category_ID)
    REFERENCES Category(Category_ID),

  FOREIGN KEY (Seller_ID)
    REFERENCES Seller(Seller_ID)
);
```

5. Orders Table

```
CREATE TABLE Orders (
   Order_ID INT PRIMARY KEY,
   Customer_ID INT,
   Order_Date DATE,
   Order_Status VARCHAR(50),
   Total_Amount DECIMAL(10,2),

   FOREIGN KEY (Customer_ID)
      REFERENCES Customer(Customer_ID)
);
```

6. Order Details Table

```
CREATE TABLE Order_Details (
   Order_Detail_ID INT PRIMARY KEY,
   Order_ID INT,
   Product_ID INT,
   Quantity INT,
   Unit_Price DECIMAL(10,2),
   Subtotal DECIMAL(10,2),

   FOREIGN KEY (Order_ID)
      REFERENCES Orders(Order_ID),

   FOREIGN KEY (Product_ID)
      REFERENCES Product(Product_ID)
);
```

7. Payment Table

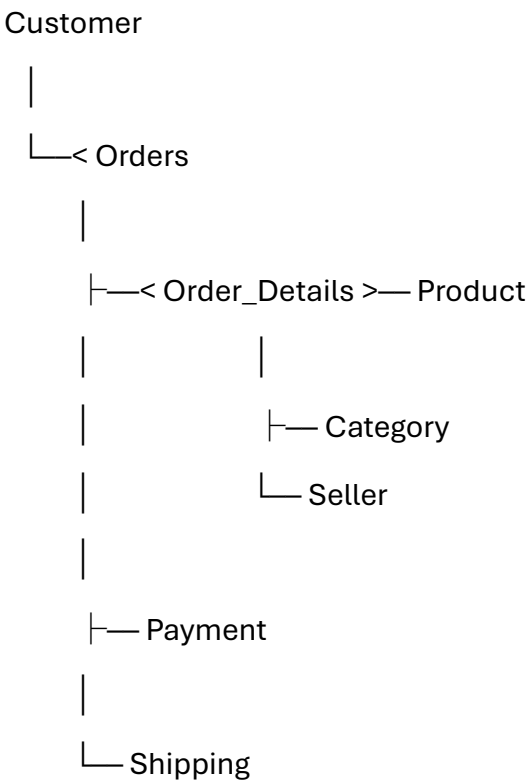
```
CREATE TABLE Payment (
```

```
    Payment_ID INT PRIMARY KEY,
    Order_ID INT,
    Payment_Method VARCHAR(50),
    Amount DECIMAL(10,2),
    Payment_Date DATE,
    Payment_Status VARCHAR(50),

    FOREIGN KEY (Order_ID)
        REFERENCES Orders(Order_ID)
);
```

8. Shipping Table

```
CREATE TABLE Shipping (
    Shipping_ID INT PRIMARY KEY,
    Order_ID INT,
    Shipping_Address VARCHAR(255),
    Tracking_Number VARCHAR(100),
    Delivery_Status VARCHAR(50),

    FOREIGN KEY (Order_ID)
        REFERENCES Orders(Order_ID)
);
```

Complete Relationship Flow



This SQL schema follows the entities, attributes, primary keys, foreign keys, and relationships defined in your E-Commerce Order Management System.

Conclusion

Thus, the ER Diagram has been successfully converted into a relational schema. **Primary Keys** uniquely identify each record, while **Foreign Keys** maintain relationships and ensure data integrity between Customer, Product, Order, Payment, and Shipping tables. The Order_Details table connects Orders and Products and stores the individual items included in each order.